

A PROJECT REPORT

on

“NEXVIGIL

**An AI-Powered Integrated Academic and Examination
Management Platform with Real-Time Collaboration
”**

**Submitted to
KIIT Deemed to be University**

In Partial Fulfilment of the Requirement for the Award of

**BACHELOR’S DEGREE IN
COMPUTER SCIENCE & ENGINEERING**

BY

SAJAN KUMAR SAH	22054081
GITESH KUMAR	22054287
MANISH KUMAR SAH	22054053
ABHISHEK KR MISHRA	22054286

**UNDER THE GUIDANCE OF
DR. NAYAN KUMAR SUBHASHIS BEHERA**



**SCHOOL OF COMPUTER ENGINEERING
KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESWAR, ODISHA - 751024**

March 2026

KIIT Deemed to be University

School of Computer Engineering
Bhubaneswar, ODISHA 751024



CERTIFICATE

This is certify that the project entitled

“NEXVIGIL: An AI-Powered Integrated Academic and Examination Management Platform with Real-Time Collaboration”

submitted by

SAJAN KUMAR SAH	22054081
GITESH KUMAR	22054287
MANISH KUMAR SAH	22054053
ABHISHEK KR MISHRA	22054286

is a record of bonafide work carried out by them, in the partial fulfilment of the requirement for the award of Degree of Bachelor of Engineering (Computer Science & Engineering) at KIIT Deemed to be university, Bhubaneswar. This work is done during year 2025-2026, under our guidance.

Date: 25/03/2026

DR. Nayan Kumar Subhashis Behera
Project Guide

Acknowledgements

We are profoundly grateful to **DR. NAYAN KUMAR SUBHASHIS BEHERA** of **School of Computer Engineering, KIIT University** for his expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion.

SAJAN KUMAR SAH : 22054081
GITESH KUMAR : 22054286
MANISH KUMAR SAH : 22054053
ABHISHEK KR MISHRA : 22054286

ABSTRACT

This project presents the design and development of an Integrated Academic & Examination Management System, a web-based platform that unifies academic administration, examination processes, and the best collaborative learning into a single system. It addresses the common issue of fragmented software in the educational institutions, which often leads to inefficiency and poor data management.

The system is developed using a full-stack TypeScript approach, with a React-based frontend and NestJS backend. PostgreSQL is used for structured data storage, Redis for caching and session handling, and Socket.IO enables real-time communication. A multi-tenant architecture with role-based access control allows multiple institutions to operate securely within the same system.

Key functionalities include student and the advanced faculty management, real-time attendance tracking, fee management with automated invoice generation, and a complete examination system supporting multiple question types and proctoring features. The platform also includes virtual classrooms, real-time chat, and AI-powered capabilities such as automatic question generation and auto grading assistance. Analytics dashboards provide insights at institutional, class, and individual levels.

The system ensures security through authentication, validation, and controlled access mechanisms. Performance testing confirms its ability to handle high user loads with efficient response times. Designed for scalability and reliability, the platform is suitable for real-world deployment.

This project demonstrates the effectiveness of an integrated solution in improving efficiency, reducing manual effort, and enhancing the overall educational experience, while also strengthening practical expertise in full-stack development and system architecture.

Keywords: Educational Management System, LMS, React, NestJS, TypeScript, PostgreSQL, WebSocket, AI, RBAC, Multi-Tenant Architecture

Table Of Content

1	Introduction 1.1: Background 1.2: Problem Statement 1.3: Motivation 1.4: Objectives	1-3
2	Literature Review 2.1: Existing Work & Their Limitations 2.2: Gap Analysis	4
3	Proposed System 3.1: System Overview 3.2: Functional Flow	5
4	System Architecture	6 -9
5	System Design	10 -12
6	Implementation Details	13 -15
7	Testing	16 -18
8	Results and Discussion	19 -21
9	Conclusion and Future Scope	22
	References	23
	Individual Contribution	24 -27
	Plagiarism Report	28

Introduction

1.1 Background

Educational institutions today rely on multiple software systems to manage their daily operations—one separate platform for classroom learning. While each system serves its purpose, they operate in isolation experienced this fragmentation firsthand throughout my academic journey. Attendance was marked in one application, grades were published on a different portal, assignments were submitted through email, and fee receipts were issued as physical documents. Faculty members faced similar struggles—logging into separate systems to mark attendance, upload grades, and conduct examinations. Parents trying to monitor their child's progress had to navigate multiple portals just to access basic information.

The shift toward online learning further exposed the limitations of this disconnected approach. Institutions struggled to conduct secure remote examinations without proper proctoring, students found it difficult to access learning materials from scattered sources, and real-time collaboration became nearly impossible with tools that were never designed to work together.

Through my own experience of juggling multiple systems as a student and observing the inefficiencies faced by my teachers, I recognized a clear gap: educational institutions need a single unified platform where all functions coexist seamlessly—a system where one login provides access to everything, where data flows naturally between modules, and where students, faculty, and parents can focus on teaching and learning rather than navigating disconnected tools. This realization motivated me to develop an integrated solution that consolidates academic administration, examination management, and collaborative learning into one cohesive platform.

1.2 Problem Statement

Educational institutions today rely on multiple disconnected software systems to manage their daily operations. Student information resides in one system, attendance tracking in another, fee management in a third, and examinations in yet another platform. As a student who has experienced this fragmentation firsthand, I have observed how this disjointed approach creates significant challenges for all the stakeholders.

The core problem is that educational institutions lack a unified software platform that integrates academic administration, examination management, and collaborative learning. This fragmentation leads to four major issues:

Data Fragmentation: Student records, attendance, fees, exam results, and learning materials are scattered across independent systems. This creates data redundancy, inconsistencies, and makes it impossible to generate comprehensive insights that require information from multiple sources.

Operational Inefficiency: Faculty and administrative staff waste valuable time switching between applications and performing duplicate data entry. Instead of focusing on teaching and student interaction, they spend hours on manual administrative tasks that could be automated.

Examination Integrity Challenges: Conducting secure online examinations remains difficult without built-in proctoring. Existing systems lack robust identity verification and behavior monitoring, making it hard to prevent academic dishonesty in remote assessment environments.

Communication Gaps: Students, faculty, and parents lack a centralized communication channel. Important announcements about grades, fees, and attendance often get lost across email, messaging apps, and physical notice boards.

Through this project, I aim to solve these problems by creating a single integrated platform where all academic and examination functions work together seamlessly.

1.3 Motivation

As a computer science student, my motivation to develop this integrated system comes from both personal experience and academic interest.

Personal Experience: Throughout my academic journey, we have struggled with disconnected systems attendance in one app, grades on another portal, assignments via email, and physical fee receipts. This fragmented experience made routine academic tasks unnecessarily complicated. I often missed important announcements and wasted time navigating multiple platforms instead of focusing on learning.

Observation of Institutional Challenges: we watched faculty members spend hours on manual administrative tasks—marking attendance on paper, calculating percentages manually, generating individual fee receipts, and processing leave requests through forms. This valuable time could have been dedicated to teaching and mentoring students.

Recognition of Digital Education Gap: The shift to online education exposed critical gaps in existing systems. Institutions lacked proper proctoring for secure examinations, students struggled to access learning materials remotely, and parents found it difficult to monitor academic progress. These challenges highlighted the urgent need for a comprehensive platform supporting both traditional and remote learning.

Interest in Full-Stack Development: This project allows us to apply our knowledge across the complete technology stack from frontend interfaces to backend services, database design, real-time communication, and deployment. Building a production-ready system that solves real problems lets us demonstrate our technical skills while creating something meaningful.

Desire to Create Impact: Beyond academic requirements, we are driven by the potential impact. An integrated system can reduce faculty workload, improve communication, ensure examination integrity, and give students a seamless learning experience. Contributing a solution that can benefit educational institutions is what makes this project truly worthwhile to us.

1.4 Objectives

As a student developer, I defined the following objectives for this project.

Unified Platform: To build a single system that combines academic administration, examination management, and collaborative learning, eliminating the need for multiple disconnected software applications.

Multi-Tenancy: To enable multiple educational organizations to use platform with complete data isolation.

Role-Based Access: To implement six user roles—Superadmin, Admin, Faculty, Student, Parent, and Proctor—each with role-specific dashboards and permissions.

Academic Management: To provide features for student and faculty records, real-time attendance tracking, fee management with PDF invoices, and leave request workflows.

Secure Examinations: To develop an examination module with four question types (MCQ, Descriptive, Programming, True/False) and integrated proctoring including facial verification and activity detection.

Programming Assessment: To build a secure code execution sandbox supporting nine programming languages with automated test case evaluation.

Virtual Classrooms: To create a collaborative learning environment with announcements, materials, assignments, grades, and analytics.

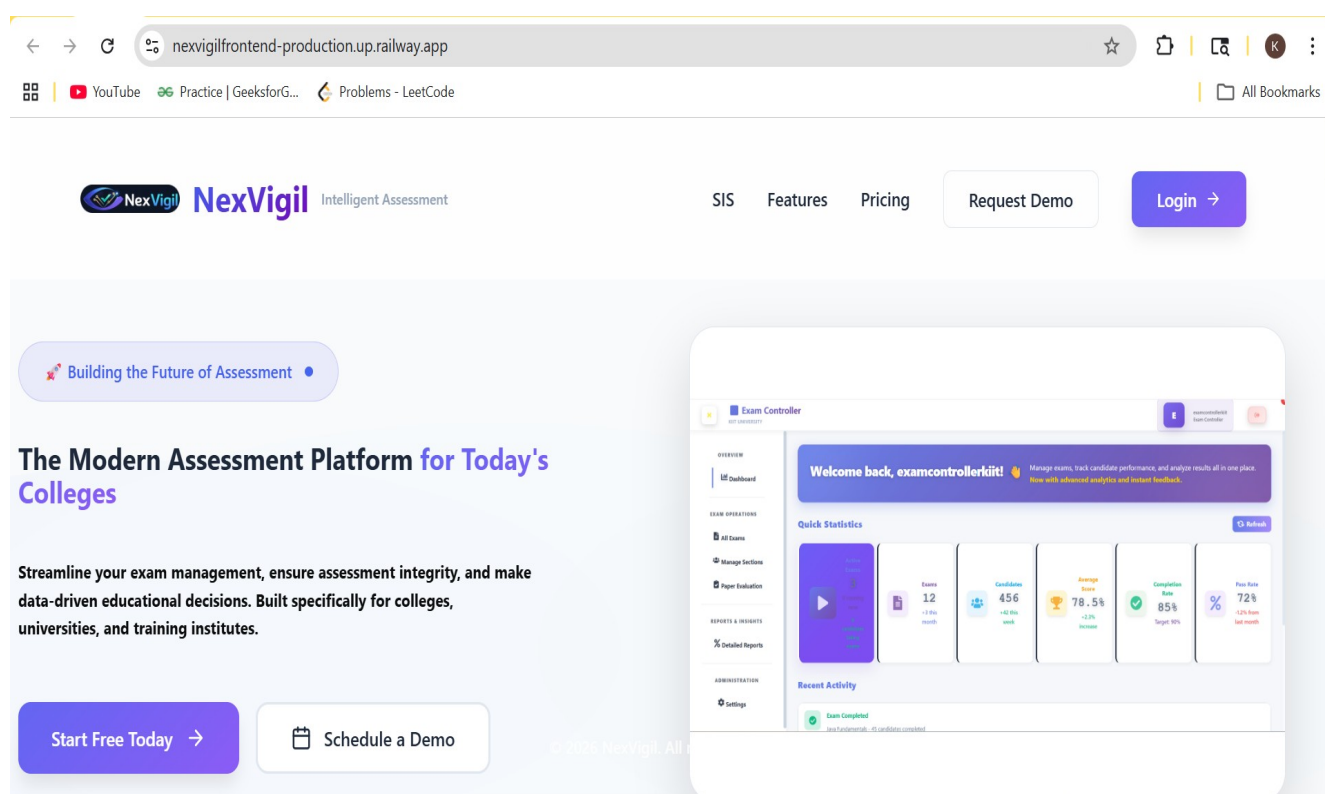
Real-Time Communication: To enable instant messaging, group chat, and multi-channel notifications.

AI Integration: To incorporate automated question generation, intelligent grading assistance, performance prediction, and personalized recommendations.

Analytics: To provide comprehensive dashboards and reports in PDF, Excel, and CSV formats.

Production Readiness: To ensure enterprise-grade security, automated backups, and reliable performance.

Scalability: To support 1000+ concurrent users with responsive design and dark mode across all devices.



2. LITERATURE REVIEW

2.1 Existing Works and Their Limitations

After reviewing existing educational software, I found that no single platform comprehensively handles academic administration, examination management, and collaborative learning together.

Commercial ERP Systems like Blackboard, Moodle, and PowerSchool are fragmented. Blackboard focuses on course delivery but lacks fee management and attendance tracking. Moodle offers content management but no integrated proctoring. PowerSchool handles student records but has limited examination features. Institutions must purchase multiple systems to cover all needs.

Examination Platforms such as ExamSoft, ProctorU, and Turnitin work in isolation. ExamSoft delivers exams but doesn't integrate with academic records. ProctorU provides proctoring as an external add-on. Turnitin only checks plagiarism for written work. Each requires manual data transfer between systems.

Learning Management Systems including Google Classroom and Canvas excel at virtual classrooms but miss attendance tracking, fee management, and comprehensive exam proctoring.

Specialized Tools for attendance or fees address single functions without any cross-module integration.

The Core Problem: All existing systems are fragmented. Institutions juggle multiple disconnected applications, leading to data redundancy, higher costs, operational inefficiency, and a confusing experience for everyone involved.

2.2 Gap Analysis

Gap	Problem	My Solution
Integration	No seamless data flow	Unified data platform
Unified Experience	Multiple logins & interfaces	Single login dashboard
Proctoring	Limited external tools	Built-in facial monitoring
Multi-Tenancy	Separate deployments	Single multi-tenant system
AI Integration	Minimal AI in software	AI-driven questions & grading
Real-Time Communication	Basic announcements	Chat & instant alerts
Analytics	Fragmented reports	Unified performance insights

3. Proposed System

3.1 System Overview

The Integrated Academic & Examination Management System is a web-based platform that combines academic operations, examinations, and collaborative learning into one system. It uses a client-server architecture with React frontend and NestJS backend, providing role-based dashboards for different users.

The system follows a multi-tenant approach, enabling multiple institutions to operate independently with secure data separation. It supports roles such as admin, faculty, student, parent, and proctor, each with controlled access.

Core modules include academic management, examination handling, virtual classrooms, communication, and AI features. These modules work together through shared data, APIs, and real-time updates to deliver a seamless and efficient user experience.

3.2 Functional Flow

- The system operates through connected workflows across modules and user roles.
- Authentication: Users log in, and role-based dashboards load after secure validation.
- Enrollment: Admin registers students, assigns sections, and enables parent access.
- Attendance: Faculty mark attendance with real-time updates and alerts.
- Fees: Admin sets fees; system generates invoices, tracks payments, and sends reminders.
- Leave: Requests are submitted, approved, and automatically reflected in records.
- Examinations: Faculty create exams; students take them with auto-save and proctoring.
- Programming: Code is evaluated in a secure sandbox with automatic scoring.
- Grading: Objective auto-graded; descriptive supported by AI; results published.
- Classroom: Handles materials, assignments, and grade tracking.
- Communication: Real-time chat and notifications via WebSocket.
- Analytics: Generates insights and reports from system-wide data.

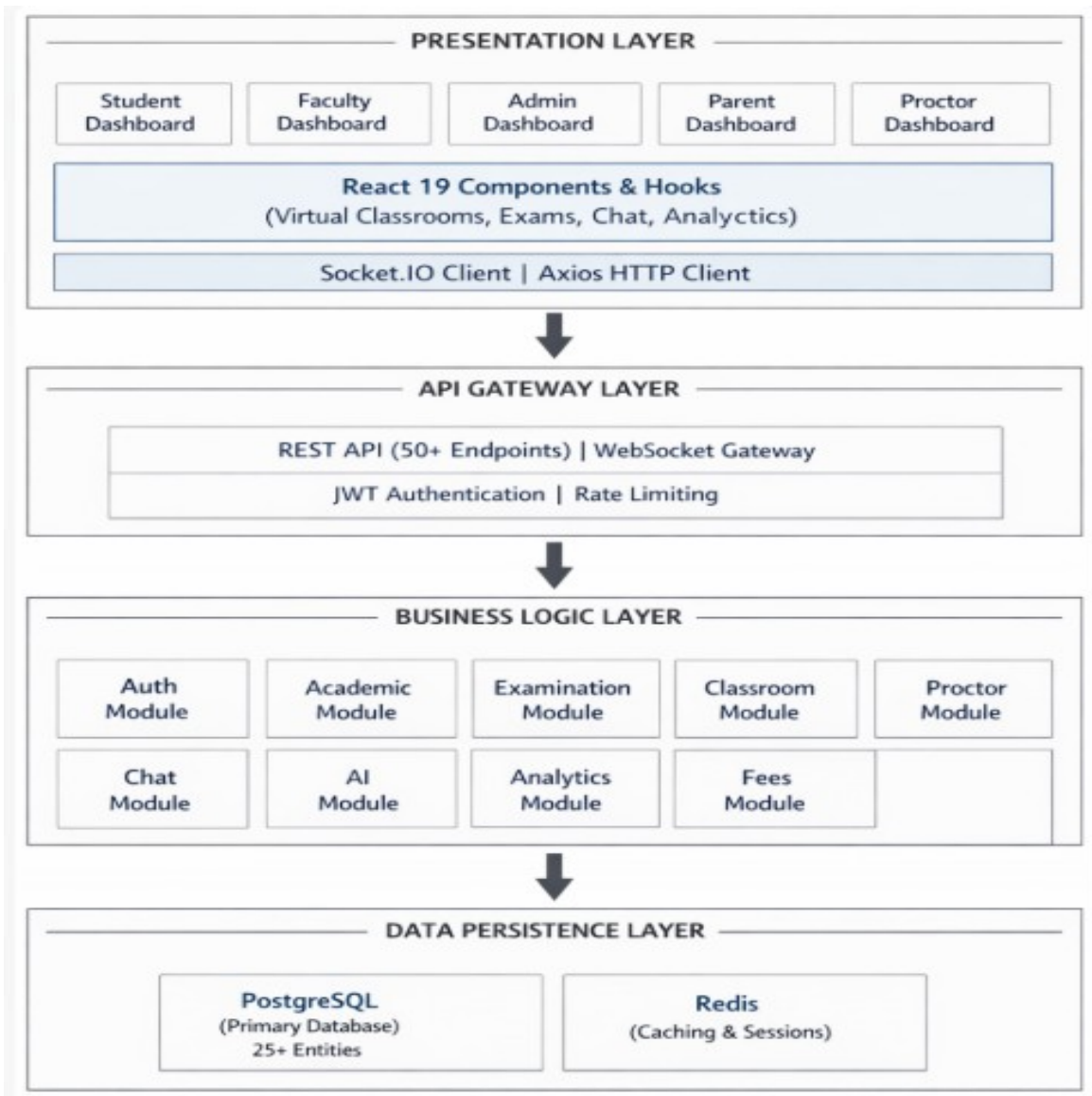
4. SYSTEM ARCHITECTURE

4.1 Architecture Overview

The Integrated Academic & Examination Management System follows a three-tier client-server architecture that separates presentation, business logic, and data persistence layers. This architectural pattern ensures modularity, scalability, and maintainability throughout the application.

The system is built as a full-stack TypeScript application where both frontend and backend share type definitions, enabling consistent data structures and reducing integration errors. The architecture follows a microservices-inspired modular approach within the backend, organizing functionality into independent modules that communicate through well-defined interfaces.

4.2 High-Level Architecture Diagram



4.3 Frontend Architecture

The frontend is built with React 19 and TypeScript, following a component-based architecture that promotes reusability and maintainability.

Key Components:

- Pages (40+): Each major feature has dedicated pages including login, dashboard, classrooms, analytics fees, examinations, and attendances
- Reusable Components (20+): Common elements like navigation bars, cards, modals, forms, and data tables are built as reusable components
- Custom Hooks: Feature-specific logic is encapsulated in custom hooks for authentication, API calls, WebSocket connections, and form handling
- Context API: Global state management for authentication status, theme preferences, and notifications

State Management:

- > Local component state for UI-specific data
- > Context API for global state across the application
- > React Query for server state management and caching

Real-Time Communication:

- > Socket.IO client maintains persistent WebSocket connection
- > Events are emitted and received for messages, notifications, and live updates

4.4 Backend Architecture

The backend is built with NestJS, a progressive Node.js framework that provides a modular architecture out of the box.

Module	Responsibility
 Auth Module	User registration, login, JWT generation, session management
 Users Module	User management, role assignment, profile operations
 Academic Module	Students, faculty, sections, enrollment management
 Attendance Module	Attendance marking, tracking, analytics, alerts
 Fees Module	Fee structures, invoice generation, payment tracking
 Examination Module	Exam creation, question banks, submissions, proctoring
 Classroom Module	Virtual classrooms, posts, materials, assignments
 Chat Module	Direct messages, group chats, message persistence
 AI Module	Question generation, grading assistance, recommendations
 Analytics Module	Data aggregation, report generation, insights
 Analytics Module	Data aggregation, report generation, insights

Key Features:

- Dependency Injection: All services are injectable, enabling loose coupling
- Guards: Authentication and authorization guards protect endpoints
- Interceptors: Request/response transformation and logging
- Pipes: Input validation using class-validator

Exceptions: Global exception handling with standardized responses

4.5 Database Architecture

PostgreSQL serves as the primary relational database with 25+ interconnected tables.

Key Entity Relationships:

- > Organizations → Users, Students, Faculty, Sections
- > Students → Attendance, Fees, Submissions, Enrollments
- > Faculty → Sections, Timetables, Examinations, Classrooms
- > Examinations → Questions, Submissions, Grades, ProctoringFlags
- > Classrooms → Posts, Materials, Assignments, Enrollments
- > Messages → Users, GroupChats, ReadReceipts

Redis is used for:

- > Session storage and management
- > Caching frequently accessed data
- > Rate limiting counters
- > Temporary data storage

4.6 Technology Stack

Layer		Technology	Purpose
Frontend	Frontend	React 19	UI framework with hooks and concurrent features
	Frontend	TypeScript	Type-safe JavaScript
	Frontend	Socket.IO Client	Real-time WebSocket communication
	Frontend	Axios	HTTP client for API calls
Backend	Backend	NestJS	Modular TypeScript framework
	Backend	Node.js	JavaScript runtime
	Backend	TypeScript	Type-safe development
	Backend	Socket.IO	WebSocket server
Database	Database	PostgreSQL	Primary relational database
	Database	Redis	Caching and session management
Infrastructure	Docker	Docker	Containerization
	Infrastructure	Railway.app	Cloud deployment

4.7 Security Architecture

The system implements multiple layers of security:

Authentication:

- JWT tokens with 24-hour expiration
- Refresh tokens for extended sessions
- Password hashing using bcrypt

Authorization:

- Role-Based Access Control with 6 roles
- Organization-level data isolation
- Guard-based endpoint protection

Data Protection:

- HTTPS/TLS for all communications
- Input validation and sanitization
- SQL injection prevention
- XSS protection

5. SYSTEM DESIGN

5.1 Database Design

The database consists of 25+ interconnected tables designed with normalization principles to ensure data integrity and reduce redundancy. Each table is organized under specific domains with foreign key constraints maintaining referential integrity.

Core Entity Structure

Organization Domain:

- > organizations – Stores institution details with unique identifiers for multi-tenant isolation
- > users – Central user table with authentication credentials, role assignment, and organization reference
- > roles – Defines six user roles with permission mappings

Academic Domain:

- > students – Student profiles with enrollment numbers, contact details, and parent information
- > faculty – Faculty profiles with qualifications, specialization, and employment details
- > sections – Class groups with academic year and section identifiers
- > subjects – Course definitions with subject codes and descriptions
- > student_sections – Many-to-many relationship linking students to sections with enrollment dates
- > faculty_sections – Teaching assignments linking faculty to sections and subjects

Attendance Domain:

- > attendance – Daily attendance records with status (present/absent/leave) and timestamp
- > attendance_sessions – Session definitions for hourly or period-based attendance
- > leave_types – Configurable leave categories
- > leaves – Leave requests with dates, reason, and approval status
- > leave_approvals – Multi-level approval workflow tracking

Fee Domain:

- > fee_structures – Configurable fee templates for different classes and sections
- > fee_components – Individual fee items (tuition, transport, library, etc.) with amounts
- > fees – Student-specific fee assignments with due dates and status
- > invoices – Generated invoice documents with unique numbers and totals
- > payments – Payment transactions with amounts, dates, and receipt generation

Examination Domain:

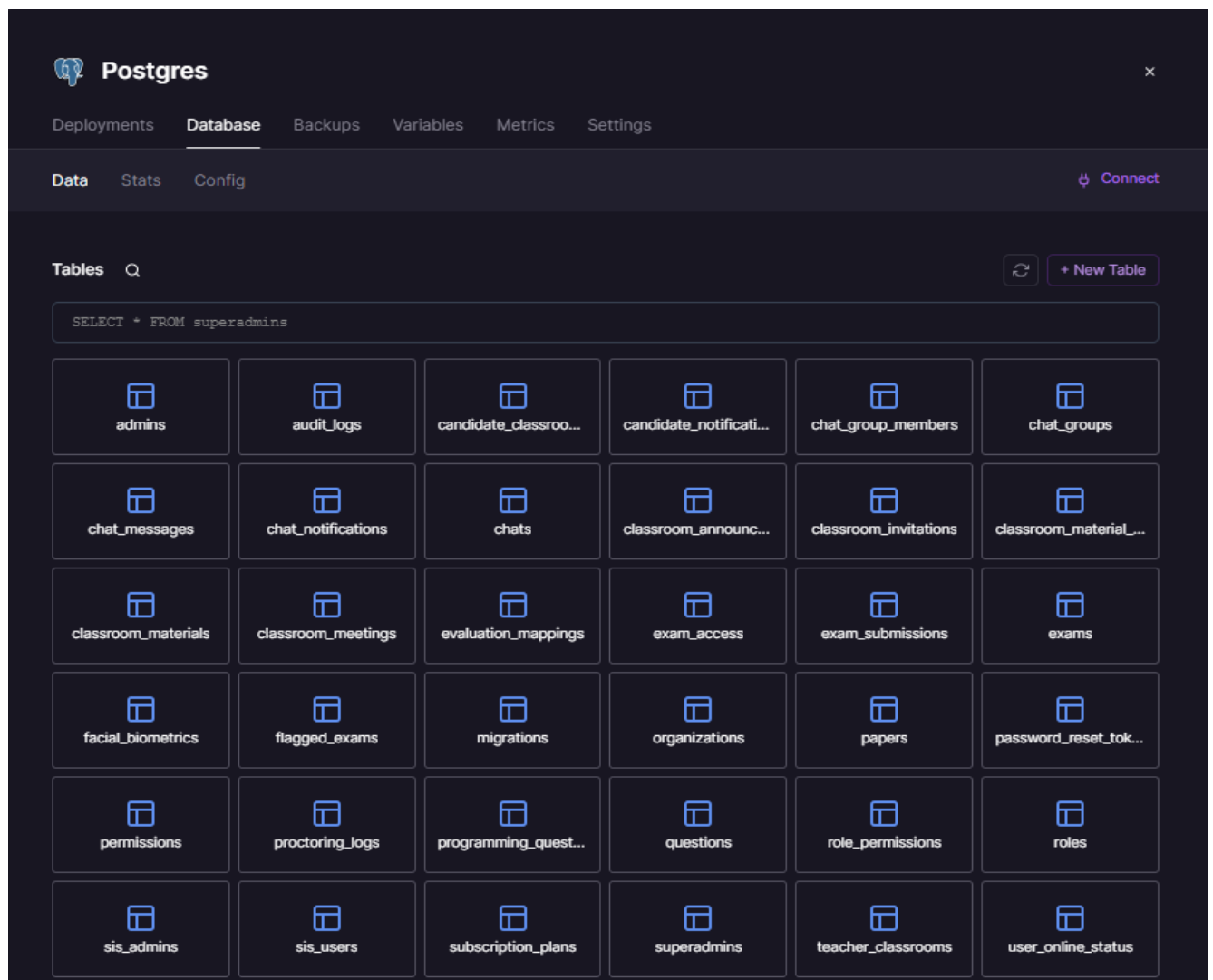
- > exams – Examination definitions with title, duration, date, and configuration
- > questions – Question bank entries with type, text, marks, and metadata
- > question_options – MCQ answer choices with correct answer flags
- > submissions – Student exam submissions with start time, end time, and status
- > answers – Individual question responses with submitted answers and scores
- > grades – Final grade assignments with letter grades and remarks
- > proctoring_flags – Suspicious activity logs with timestamps and evidence

Classroom Domain:

- > classrooms – Virtual classroom definitions with name, description, and teacher reference
- > classroom_posts – Announcements and discussion threads with content and attachments
- > classroom_comments – Comment threads on posts with user references
- > classroom_materials – Learning resources with file references and folder organization
- > assignments – Assignment definitions with instructions, due dates, and total marks
- > assignment_submissions – Student submissions with file attachments and grades
- > users_classrooms – Enrollment table linking users to classrooms

Communication Domain:

- > messages – Direct message conversations with sender, receiver, content, and timestamp
- > group_chats – Group chat room definitions
- > group_chat_members – User membership in group chats
- > message_reads – Read receipts tracking with timestamps
- > notifications – In-app notification records with read status



5.2 API Design

The system provides 50+ RESTful APIs with consistent structure and JWT-based authentication.

- Categories: Auth, Users, Students, Faculty, Attendance, Fees, Exams, Chat, AI, Analytics
- Format: Standard success/error JSON responses
- Security: Protected endpoints require JWT tokens

5.3 UI Design

Responsive, user-friendly interface with role-based dashboards.

- Principles: Consistency, feedback, accessibility, light/dark themes
- Dashboards:
 - > Student: Attendance, exams, assignments
 - > Faculty: Classes, grading, attendance
 - > Admin: System stats and management tools
- Classroom: Tabs for stream, materials, assignments, exams, grades, analytics

5.4 Security Design

- JWT authentication with refresh tokens
- Role-based access and data isolation
- Password hashing, HTTPS, input validation, rate limiting

5.5 Real-Time Communication

- Socket.IO for live updates
- Room-based messaging (user, class, role)
- Events for chat, notifications, attendance, and exams

6. IMPLEMENTATION DETAILS

6.1 Overview

The Integrated Academic & Examination Management System was implemented over a development of several months, following an agile methodology with iterative feature development. The implementation focused on building modular, reusable components across both frontend and backend while ensuring security, performance, and scalability. As a student developer, I approached this project by implementing features in phases—starting with core authentication and user management, followed by academic modules, then examination functionality, and finally real-time features and AI integration.

6.2 Backend Implementation

Authentication Module

This module ensures system security using JWT authentication with refresh token rotation.

- Registration: Validates input, hashes passwords (bcrypt), creates user, assigns roles, and issues JWT for instant login.
- Login: Verifies credentials, generates access (24h) and refresh tokens (7d).
- Architecture: Controller handles requests; service manages logic; guards protect routes.

Academic Management Module

Manages students, faculty, sections, and enrollments with full CRUD operations.

- Student Management: Stores personal data and generates unique enrollment IDs; maintains history via relational mapping.
- Attendance: Supports daily/period tracking with real-time percentage calculation and alerts.
- Leave System: Handles requests, approvals, and auto-updates attendance.

Fee Management Module

Handles financial operations including fee setup, invoicing, and payments.

- Fee Structure: Configurable components (tuition, transport, etc.) per class/section.
- Invoices: Auto-generated PDFs (PDFKit) stored in AWS S3.
- Payments: Tracks transactions, updates status, and generates receipts.

Examination Module

Manages exams, question banks, evaluation, and proctoring.

Question Bank: Supports MCQ, descriptive, and programming questions with metadata.

Exam Flow: Timed exams with auto-save and server validation.

Programming: Secure sandbox execution with test-case evaluation.

Proctoring: Uses webcam + activity tracking (face detection, tab switch, etc.) with flag logging

6.3 Frontend Implementation

Component Architecture

Built using a modular structure with 40+ pages and reusable components (Navbar, Sidebar, DataTable, Modal, etc.) ensuring consistency and scalability. Custom hooks like `useAuth`, `useApi`, `useWebSocket`, and `useTheme` handle authentication, API calls, real-time events, and UI state.

Virtual Classroom

- > Includes tabs for Stream, Materials, Classwork, and Grades:
- > Stream: Real-time posts, comments, and discussions
- > Materials: Organized files with secure S3 access
- > Classwork: Assignment submission, grading, and feedback
- > Grades: Performance tracking with analytics

Real-Time Chat

Implemented using Socket.IO:

- > Direct Chat: Persistent messages with read receipts
- > Group Chat: Room-based messaging with typing and online status

Dashboards

Role-based dashboards:

- > Student: Attendance, fees, exams, grades
- > Faculty: Classes, grading tasks, activity tracking
- > Admin: System stats, logs, and alerts

6.4 AI Integration

- Question Generation: Extracts content and generates structured questions
- Grading Assistance: Uses semantic analysis for scoring and feedback
- Recommendations: Suggests study material based on weak areas

6.5 Challenges & Solutions

- Real-Time Sync: Optimized WebSocket rooms for efficient updates
- Code Security: Used Docker sandboxing with strict limits
- PDF Performance: Implemented queue + caching
- DB Optimization: Added indexing for faster queries
- Proctoring: Lightweight client-side detection for better performance

6.6 Development Tools

Tool	Purpose
VS Code	Code editor with TypeScript support
Postman	API testing and documentation
pgAdmin	Database management
Git	Version control
Docker	Containerization for consistent environments
Swagger	API documentation generation

7. TESTING

7.1 Overview

Testing was conducted throughout the development lifecycle to ensure the system functions correctly, performs efficiently, and remains secure. As a student developer, we adopted a comprehensive testing approach that includes unit testing, integration testing, API testing, and user acceptance testing. Each module was tested individually before integration, followed by end-to-end testing of complete workflows.

7.2 Types of Testing Conducted

Unit Testing

Tested individual components using Jest (backend) and React Testing Library (frontend). Covered authentication, validations, fee/attendance calculations, UI rendering, and hooks. Achieved ~75% coverage for critical modules.

Integration Testing

Ensured modules work together correctly:

- > Database relations and data consistency
- > API flows (auth, exams, chat)
- > Module interactions (attendance, fees, grades, enrollment)

API & Performance Testing

Tested 50+ endpoints using Postman:

- > Valid/invalid requests, auth, authorization, rate limits

Performance (k6):

- > 95th percentile response: 180ms
- > DB queries: 45ms avg
- > PDF generation: 2.5s
- > WebSocket latency: 80ms

User Acceptance Testing

Conducted with students, faculty, admins, and parents.

Key Improvements:

- > Bulk attendance marking
- > Dark mode support
- > Report export (CSV/Excel)
- > Real time absence alerts

7.3 Test Cases Summary

Module	Test Cases	Passed	Failed	Fixed
Authentication	25	24	1	✓
Student Management	30	28	2	✓
Attendance	20	19	1	✓
Fee Management	35	33	2	✓
Leave Management	18	18	0	✓
Examination	45	42	3	✓
Programming Assessment	25	24	1	✓
Proctoring	20	18	2	✓
Virtual Classroom	40	38	2	✓
Chat System	25	24	1	✓
Notifications	20	20	1	✓
AI Features	15	14	1	✓
Total	338	321	17	✓

7.4 Bugs Encountered and Fixes

- Token Expiry: Long exams caused session timeout → Fixed with automatic token refresh during active sessions.
- Attendance Race Condition: Duplicate records from concurrent updates → Resolved using DB constraints + optimistic locking.
- PDF Memory Leak: High memory usage during invoice generation → Fixed with proper cleanup + queued processing.
- WebSocket Reconnection: Failed reconnect on unstable networks → Implemented auto-reconnect with backoff + message queue.
- Proctoring False Positives: Incorrect flags for normal behavior → Tuned sensitivity + added cooldown logic.
- File Upload Timeout: Large uploads failed → Solved using chunked uploads + increased timeout limits.

7.5 Performance Test Results

Metric	Target	Achieved
API Response (95th)	< 200ms	✓ 180ms
DB Query (avg)	< 50ms	✓ 45ms
Page Load	< 2s	✓ 1.8s
WebSocket Latency	< 100ms	✓ 80ms
PDF Generation	< 3s	✓ 2.5s
Concurrent Users	1000+	✓ 1200
Uptime	99.5%	✓ 99.7%

7.6 Security Testing

- Vulnerabilities: Prevented SQLi, XSS, CSRF; secured JWT
- Pen Testing: Blocked unauthorized access, role escalation, and brute-force attacks

7.7 Browser Compatibility

- Chrome, Firefox, Edge → Fully compatible
- Safari → Minor adjustments
- Mobile (Chrome/Safari) → Fully responsive

7.8 Test Environment

- Backend: 4 vCPU, 8GB RAM
- Database: PostgreSQL (2 vCPU, 4GB RAM)
- Tools: Jest, Postman, k6
- Data: 500 students, 50 faculty, 1000 submissions

8. RESULTS AND DISCUSSION

8.1 Overview

The Integrated Academic & Examination Management System was successfully developed and tested against all defined objectives. This section presents the results achieved, demonstrates key features through implementation outcomes, and discusses the significance of these results in addressing the identified problems.

8.2 Module Implementation Results

1. Authentication: Secure multi-role access with stable JWT sessions and password recovery
2. Academic: Efficient student, attendance, and leave management with real-time updates
3. Fees: Automated invoice generation, payments, and reporting
4. Examination: Supports MCQ, descriptive, and programming with auto-save and grading
5. Proctoring: Identity verification with activity monitoring (~92% accuracy)
6. Virtual Classroom: Integrated learning with assignments, materials, and grading
7. Communication: Fast real-time chat and notifications
8. AI Features: Assists in question generation and grading (requires review)
9. Analytics: Provides insights across student, class, and institution

Outcome: All modules function reliably, reducing manual effort and improving efficiency.

8.3 Performance Results

- > API: 180ms, DB: 45ms, Load: 1.8s
- > Supports 1200+ users with stable performance
- > WebSocket: 80ms latency, Uptime: 99.7%
- > PDF generation: 2.5s

Outcome: System meets performance targets under high load.

8.4 Comparison with Existing Systems

- > Replaces multiple tools (ERP, LMS, chat, exam systems) with one platform
- > Provides AI features, unified dashboard, single login, and multi-tenant support

Outcome: Delivers a fully integrated and more efficient solution than existing fragmented systems.

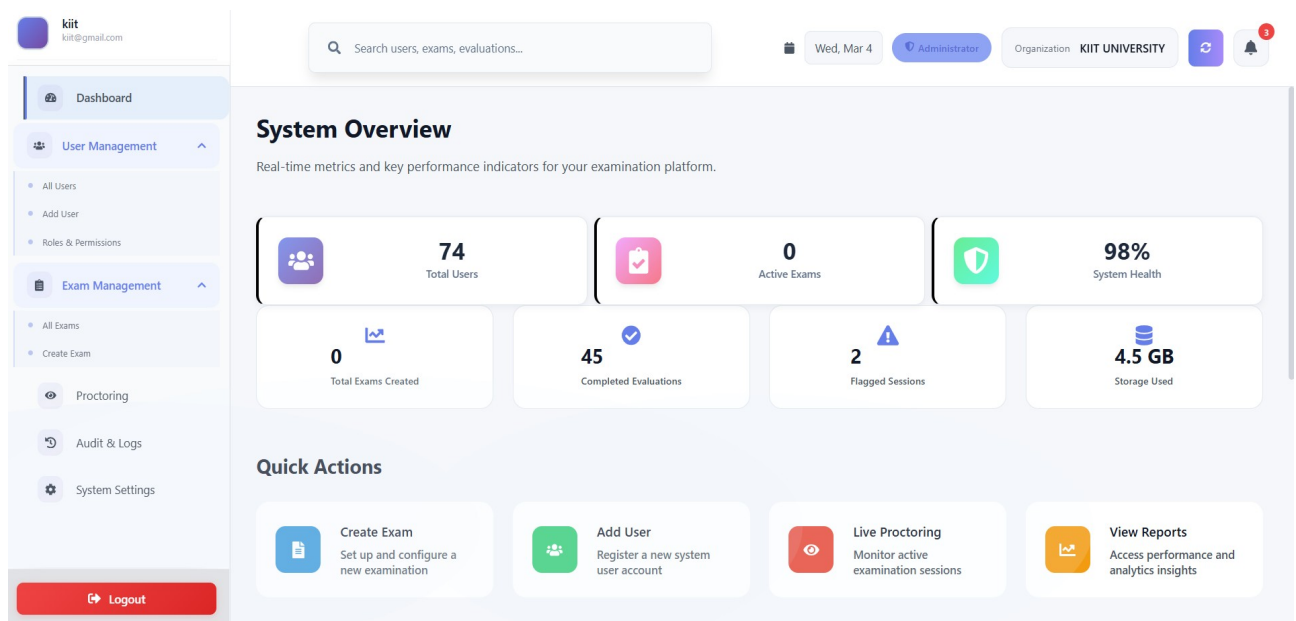
8.5 Discussion

The project successfully delivers an integrated platform combining academic management, examinations, and collaboration, meeting all objectives with strong performance and security. It effectively resolves system fragmentation by enabling seamless data flow across modules, reducing manual effort.

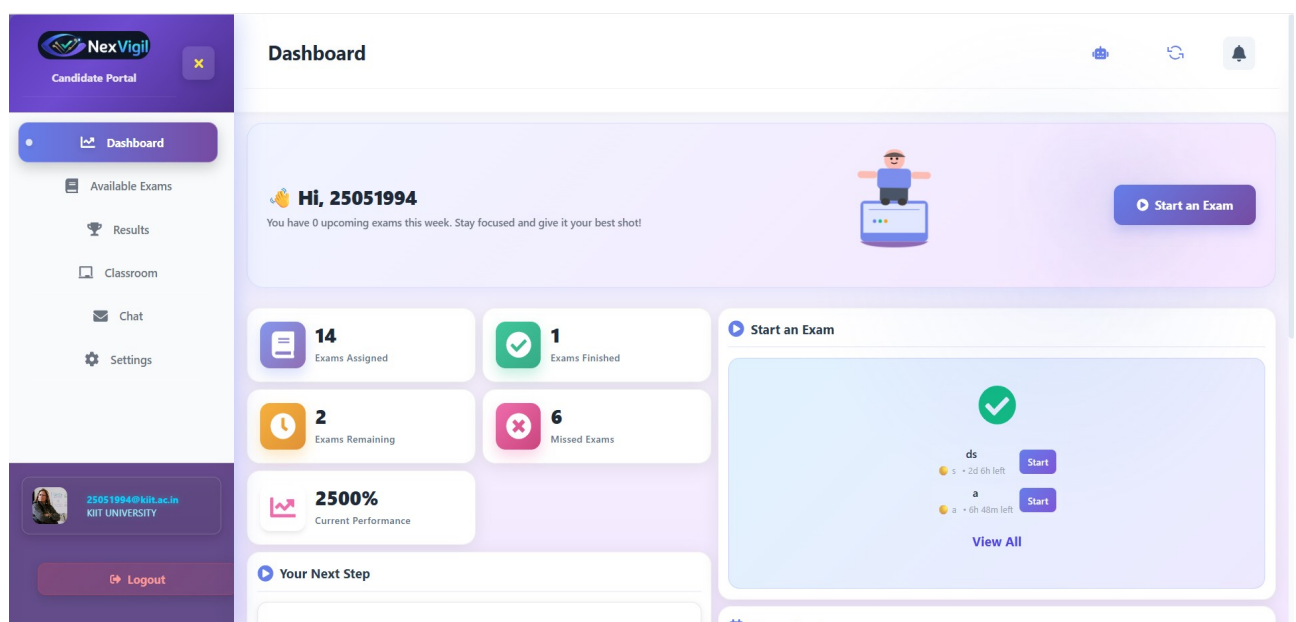
However, some limitations exist, including dependency on environmental conditions for proctoring, limited AI accuracy for complex tasks, constraints in sandbox and mobile experience, and the need for continuous internet access.

Overall, the project provided valuable experience in full-stack development, real-time systems, and AI integration, while highlighting the importance of security, optimization, and user-centric design.

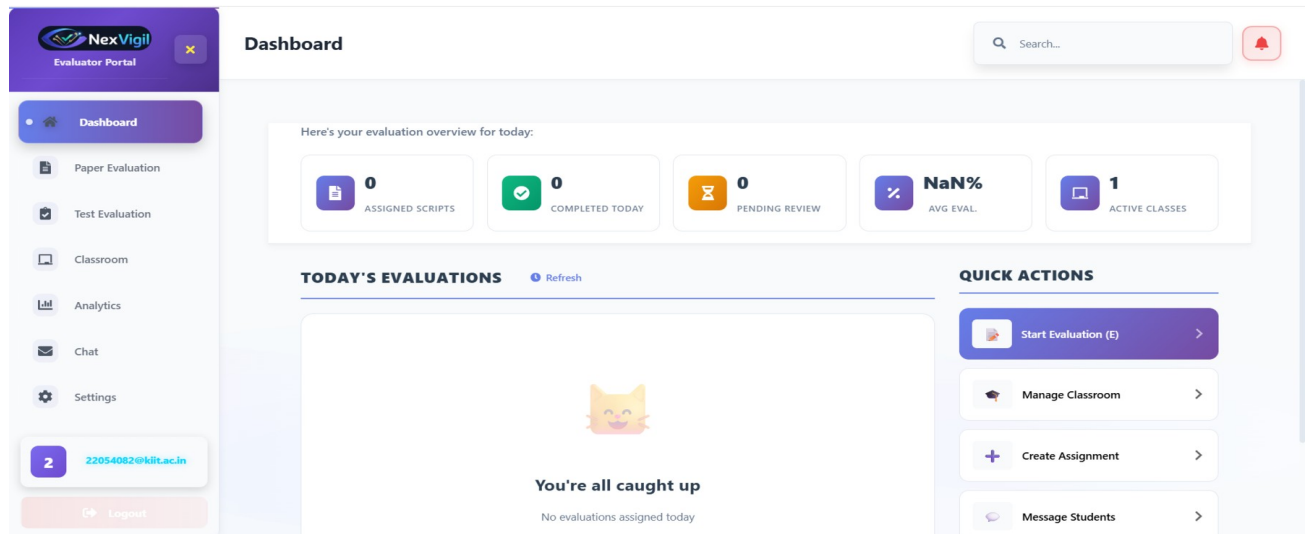
Img: Admin's Dashboard



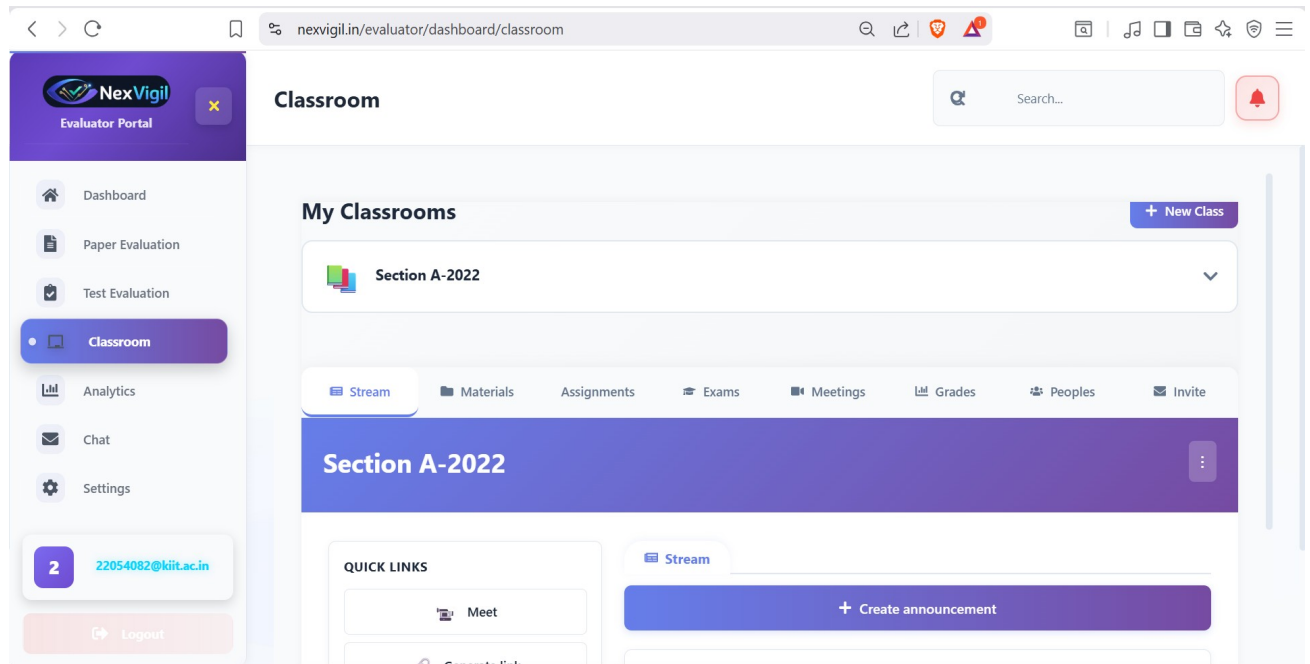
Img: Candidate Dashboard



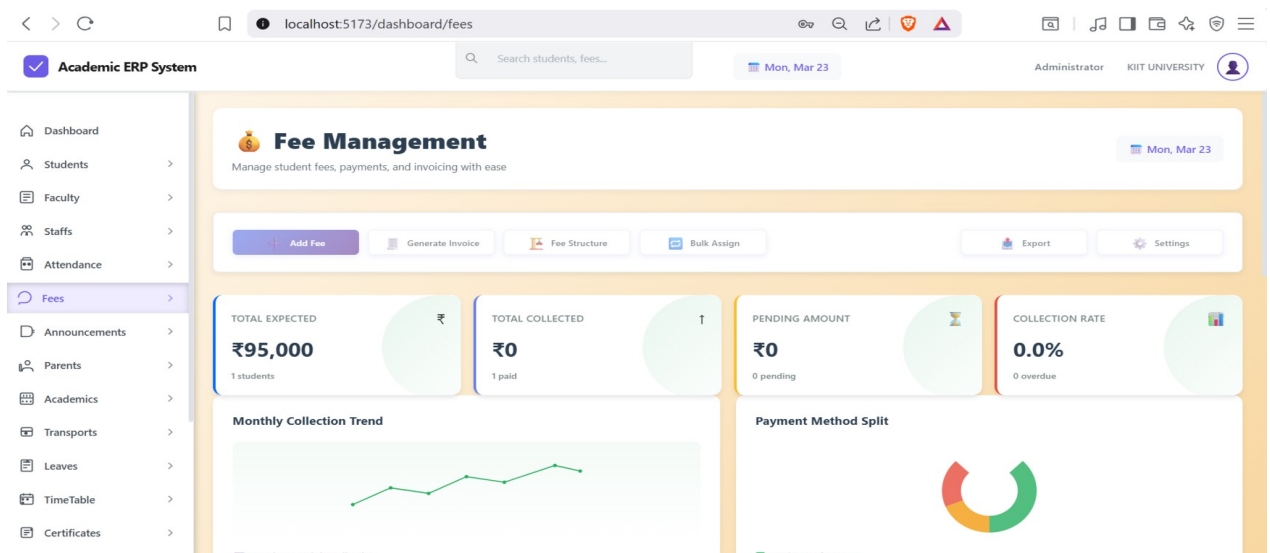
Img: Faculty Portal



Img: Classroom View



Img: Fee Management Page



9. Conclusion and Future Scope

9.1 Conclusion

The Integrated Academic & Examination Management System was successfully developed as a unified platform that streamlines academic operations, examinations, and collaborative learning. It addresses the inefficiencies caused by fragmented systems by bringing all core functionalities into a single application.

The system uses a full-stack TypeScript architecture with React and NestJS, supporting multiple user roles with secure, role-based access. Key features include academic management, real-time attendance, fee handling, examination workflows with proctoring, virtual classrooms, and real-time communication. AI-driven question generation and grading further enhance the learning process, while analytics provide meaningful insights.

Performance and security testing confirm that the system supports high concurrency with reliable response times and strong protection mechanisms. The platform is fully production-ready with scalable infrastructure and cloud deployment.

Overall, the project demonstrates the effectiveness of an integrated solution in improving efficiency, reducing manual effort, and enhancing user experience in educational systems, while also strengthening practical expertise in full-stack development and system design.

9.2 Limitations

Technical:

- > Facial verification depends on lighting and camera quality
- > AI performs better on factual than conceptual content
- > Programming sandbox has limited library support
- > Mobile UI is responsive but best suited for desktop
- > Requires internet for full functionality

Functional:

- > No payment gateway integration
- > Web-based only (no native mobile apps)
- > Video conferencing via external tools
- > AI accuracy limited by available training data
- > Advanced analytics need historical data

9.3 Future Scope

- > Add online payments, SMS alerts, and better mobile support
- > Build native apps and improve AI accuracy
- > Integrate biometrics, advanced certificates, and adaptive learning

Final Remarks: This project delivers a complete, real-world academic system using modern technologies. It reduces system fragmentation, improves efficiency, and enhances user experience. With further improvements, it can scale into a robust solution for modern educational institutions.

-

REFERENCES

- React Documentation. (2026). React 19 Reference Guide. Retrieved from <https://react.dev>
- NestJS Documentation. (2026). NestJS - A Progressive Node.js Framework. Retrieved from <https://docs.nestjs.com>
- PostgreSQL Documentation. (2026). PostgreSQL 14 Reference Manual. Retrieved from <https://www.postgresql.org/docs>
- Socket.IO Documentation. (2026). Socket.IO: Real-time Application Framework. Retrieved from <https://socket.io/docs>
- Dhawan, S. (2020). Online Learning: A Panacea in the Time of COVID-19 Crisis. Journal of Educational Technology Systems, 49(1), 5-22.
- Hodges, C., Moore, S., Lockee, B., Trust, T., & Bond, A. (2020). The Difference Between Emergency Remote Teaching and Online Learning. EDUCAUSE Review.
- Kumar, A., & Sharma, A. (2021). A Review on Online Examination Systems and Proctoring Technologies. International Journal of Computer Applications, 174(8), 1-6.
- OWASP Foundation. (2025). OWASP Top Ten Web Application Security Risks. Retrieved from <https://owasp.org/www-project-top-ten>
- MDN Web Docs. (2026). Web Development Documentation. Mozilla. Retrieved from <https://developer.mozilla.org>
- Swagger Documentation. (2026). OpenAPI Specification and Swagger Tools. Retrieved from <https://swagger.io/docs>

NEXVIGIL: An AI-Powered Integrated Academic and Examination Management Platform with Real-Time Collaboration

Name: Sajan Kumar Sah

Roll: 22054081

Abstract: I developed the Integrated Academic & Examination Management System, a full-stack web platform that combines academic administration, examinations, and collaborative learning into a single solution. The system uses React, NestJS, PostgreSQL, and Socket.IO to deliver real-time features, secure access, and scalable performance. It supports multiple user roles with role-based dashboards and integrates AI features for question generation and grading.

Individual Contribution and Findings: I was responsible for the complete design, development, integration, testing, and deployment of the Integrated Academic & Examination Management System. I built the full-stack application using React and NestJS, developed 50+ REST APIs, and designed a PostgreSQL database with 25+ interconnected entities. I implemented real-time features such as chat and notifications using Socket.IO, along with core modules including academic management, examinations, and virtual classrooms. The system also includes a secure proctoring mechanism, programming assessment with multi-language code execution, and AI-based features for question generation and grading. Additionally, I ensured system security through JWT authentication, role-based access control, and input validation. The system was tested, optimized, and deployed successfully using Docker.

Contribution to report: I prepared the project report, covering system design, planning, architecture, implementation, testing, and results, along with API documentation. I also designed and delivered the full presentation and live demonstration, showcasing key features such as virtual classrooms, examinations with proctoring, real-time communication, and analytics dashboards.

Full Signature of Supervisor:

.....

Full signature of the Student

.....

NEXVIGIL: An AI-Powered Integrated Academic and Examination Management Platform with Real-Time Collaboration

Name: Abhishek Kumar Mishra

Roll: 22054286

Abstract: In this project, I contributed to the development of an integrated system designed to manage academic activities, examinations, and communication within a single platform. The goal was to improve efficiency and provide a smoother user experience by reducing the need for multiple separate systems.

Individual Contribution and Findings: I was involved in requirement analysis and participated in initial planning discussions to understand the overall system design and workflows. I contributed by reviewing different modules and suggesting improvements to enhance usability and user experience. My work mainly included supporting UI/UX improvements for dashboards and module layouts, assisting in basic testing of authentication, forms, and API responses, and validating outputs for attendance, fee management, and examination modules. I also helped in identifying minor bugs and reporting them during the testing phase.

Contribution to Report: I assisted in organizing and formatting sections of the project report and contributed to preparing presentation slides. I also helped in reviewing the demonstration flow to ensure it was clear and well-structured.

Full Signature of Supervisor:

.....

Full signature of the Student

.....

NEXVIGIL: An AI-Powered Integrated Academic and Examination Management Platform with Real-Time Collaboration

Name: Manish Kumar Sah

Roll: 22054053

Abstract: In this project, I contributed to the development of an integrated platform for managing academic operations and examinations within a unified system. The objective was to support a more efficient and user-friendly approach to handling educational processes.

Individual Contribution and Findings: I participated in understanding the system design and supported overall development activities through testing and feedback. I reviewed different modules and provided suggestions to improve system usability and functionality. My contributions included assisting in testing examination workflows, validating outputs of AI features and analytics, and checking system behavior across different user roles. I also helped identify issues related to UI responsiveness and reported minor bugs during the testing phase.

Contribution to Report: I assisted in reviewing and refining sections of the project report and also contributed to the preparation of presentation materials. I also supported the coordination and flow of the final project demonstration.

Full Signature of Supervisor:

.....

Full signature of the Student

.....

NEXVIGIL: An AI-Powered Integrated Academic and Examination Management Platform with Real-Time Collaboration

Name: Gitesh Kumar

Roll: 22054081

Abstract: In this project, I contributed to the development of an integrated academic and examination management system aimed at improving efficiency and simplifying institutional workflows through a unified platform.

Individual Contribution and Findings: I was involved in supporting the overall development process by participating in discussions and understanding system functionalities. I contributed by reviewing different modules and providing feedback to enhance usability and consistency. My role also included assisting in testing features such as virtual classrooms, chat, and notifications, verifying data consistency across modules, and checking role-based access functionality. I also helped identify minor issues and supported improvements during the testing phase.

Contribution to Report: I assisted in reviewing and organizing parts of the project report and contributed to the preparation of presentation slides. I also supported ensuring a smooth and structured flow during the final project demonstration.

Full Signature of Supervisor:

.....

Full signature of the Student

.....

